

Flutter 구현 가이드: 멀티 에이전트 JSON 스트림의 점진적 UI 전환 아키텍처

I. 서론: "블랙박스"에서 "점진적 UI"로의 아키텍처 전환

클라이언트께서 "JSON 데이터로 반환이 되면 어떻게 또 UI로 전환을 하는데?"라고 질의하셨습니다. 이 질문은 단순한 JSON 파싱(parsing) 방법을 넘어, 제공된 AI_ANALYSIS_SYSTEM_ARCHITECTURE.pdf¹ 문서에 기술된 신규 멀티 에이전트(Multi-Agent) 아키텍처의 핵심 프론트엔드 구현 전략을 묻는 것입니다.

현재(As-Is) 아키텍처의 명확한 한계

현재 Sherpa App의 아키텍처¹는 단일 Future에 의존합니다. 이 구조는 다음과 같은 명확한 사용자 경험(UX) 및 기술적 한계를 가집니다.¹

1. "블랙박스 대기 (Black-box waiting)": 사용자는 15초에서 20초에 달하는 시간 동안¹ 로딩 화면 외에는 어떤 피드백도 받지 못합니다.
2. "All-or-Nothing" 문제: 단일 API 호출은 10개의 분석 섹션 중 하나라도 생성에 실패하면 전체 요청이 실패하는 "All-or-nothing" 특성을 갖습니다.¹
3. 경직된 UI: 최종 결과물은 단일 텍스트 문자열(analysisContent 필드)이므로¹, 차트나 체크리스트 같은 풍부한 네이티브 UI 컴포넌트로 데이터를 분리하여 렌더링하는 데 근본적인 한계가 있습니다.

미래(To-Be) 아키텍처의 핵심 목표

새로운 "멀티 에이전트 전환 고려 사항"¹ 섹션은 이러한 문제 해결을 목표로 합니다. 특히 "UX Strategy"¹는 ***점진적 3단계 디스플레이(Progressive 3-Stage Display)***를 명시적 목표로

설정했습니다. 이는 UI가 3단계(약 4초, 7초, 10초 시점)에 걸쳐 순차적으로 업데이트되어야 함을 의미합니다.¹

아키텍트 제안: **FutureBuilder**에서 **StreamBuilder**로의 전환

현재 아키텍처의 UX 한계와 새로운 아키텍처의 "점진적 UI" 요구사항을 충족시키기 위한 유일한 Flutter 네이티브 솔루션은 **FutureBuilder**에서 **StreamBuilder**로의 아키텍처 전환입니다.²

Future는 단 한 번의 비동기 응답(성공 또는 실패)만을 처리합니다. 따라서 **FutureBuilder**는 **ConnectionState.done** 상태가 될 때 단 한 번만 UI를 빌드합니다.

반면, **Stream**은 시간의 흐름에 따라 여러 개의 데이터 조각(chunk)을 지속적으로 방출(emit)할 수 있습니다.³ **StreamBuilder**는 **ConnectionState.active** 상태에서 스트림을 구독(subscribe)하며, **yield** 키워드를 통해 새로운 데이터가 전달될 때마다 UI를 자동으로 재빌드(rebuild)합니다.⁴ 이는 ¹(p. 18)의 3단계 순차 실행 흐름을 구현하는 데 가장 이상적인 패턴입니다.

본 기술 가이드는 이 전환을 위한 4단계(모델링, 수신, 바인딩, 렌더링)의 상세한 엔드투엔드(end-to-end) 구현 전략을 제공합니다.

II. 1단계: 데이터 모델링 (Data Modeling) - 구조화된 JSON 스키마의 **Dart** 클래스 변환

"점진적 3단계 디스플레이"를 구현하기 전에, 각 에이전트가 반환할 JSON 데이터의 구조를 정의해야 합니다. ¹(pp. 15-17)에 정의된 각 에이전트의 **Output Structure**를 타입-안전(type-safe)한 Dart 클래스로 변환합니다.

이는 **jsonDecode** 이후 **Map<String, dynamic>**의 키(key) 문자열을 직접 참조하여 발생하는 런타임 오류를 원천 차단하는 필수 단계입니다.⁶ 현재 **AchievementAnalysisModel**¹에서 이미 **freezed**를 사용 중이므로, 일관성을 위해 **freezed** 및 **json_serializable** 패키지를 사용합니다.⁷

통합 응답 모델 (Sealed Class)

¹(p. 18)의 3단계 순차 실행은 서로 다른 타입의 객체(`ExpertAnalysisResult`, `EmotionalCoachResult`, `LeaderAgentResult`)가 동일한 스트림을 통해 전달됨을 의미합니다. `StreamBuilder`에서 이를 효율적으로 처리하기 위해, 모든 에이전트 결과가 구현할 기본 인터페이스(추상 클래스)를 정의합니다.

Dart

```
// lib/features/analysis/models/agent_response.model.dart
import 'package:freezed_annotation/freezed_annotation.dart';

part 'agent_response.model.freezed.dart';
part 'agent_response.model.g.dart';

// (p. 25)의 폴백(Fallback) 전략을 위한 공통 인터페이스
abstract class IAgentResult {
  bool get fallback;
}

// Agent 1: 운동 전문가
@freezed
class ExpertAnalysisResult with _$ExpertAnalysisResult implements IAgentResult {
  @Implements<IAgentResult>()
  const factory ExpertAnalysisResult({
    required String patternEvaluation,
    required List<String> strengths,
    required List<String> improvements,
    required String trendAnalysis,
    required Map<String, dynamic> statistics,
    @Default(false) bool fallback,
  }) = _ExpertAnalysisResult;

  factory ExpertAnalysisResult.fromJson(Map<String, dynamic> json) =>
    _ExpertAnalysisResultFromJson(json);
}

// Agent 2: 감성 코치
@freezed
class EmotionalCoachResult with _$EmotionalCoachResult implements IAgentResult {
```

```

@Implements<IAgentResult>()
const factory EmotionalCoachResult[1]의 'personalizedTone' 등 추가 필드 생략
  @Default(false) bool fallback,
}) = _EmotionalCoachResult;

factory EmotionalCoachResult.fromJson(Map<String, dynamic> json) =>
  _$EmotionalCoachResultFromJson(json);
}

```

Leader Agent의 중첩(Nested) JSON 처리

가장 중요한 Leader Agent¹는 weeklyRoadmap과 actionPlan이라는 두 개의 중첩된 JSON 배열(List)을 포함합니다.

중첩된 JSON 객체를 올바르게 파싱하기 위해서는, 부모 클래스(LeaderAgentResult)뿐만 아니라 모든 하위 객체(WeekPlan, ActionItem)에 대해서도 fromJson 팩토리 메서드를 구현해야 합니다.¹⁰ json_serializable 패키지는 이 과정을 자동화하며, 부모 클래스에 @JsonSerializable(explicitToJson: true) 어노테이션을 추가해야 합니다.⁸

Dart

```

// lib/features/analysis/models/leader_agent.model.dart
import 'package:freezed_annotation/freezed_annotation.dart';
import 'agent_response.model.dart'; // IAgentResult 임포트

part 'leader_agent.model.freezed.dart';
part 'leader_agent.model.g.dart';

// Agent 3: 리더 에이전트
@freezed
class LeaderAgentResult with _$LeaderAgentResult implements IAgentResult {
  @Implements<IAgentResult>()
  @JsonSerializable(explicitToJson: true) // 중첩 JSON 자동 파싱 활성화
  const factory LeaderAgentResult({
    required String gapAnalysis,
    required List<WeekPlan> weeklyRoadmap,

```

```

    required Map<String, dynamic> trendVisualization, // 차트 데이터
    required List<ActionItem> actionPlan,
    @Default(false) bool fallback,
  }) = _LeaderAgentResult;

  factory LeaderAgentResult.fromJson(Map<String, dynamic> json) =>
    _$LeaderAgentResultFromJson(json);
}

// 하위 모델 1: Weekly Roadmap
@freezed
class WeekPlan with _$WeekPlan {
  @JsonSerializable() // 하위 모델에도 어노테이션 필요
  const factory WeekPlan({
    required int week,
    required String summary, // 예: "주 3회, 평균 5km"
    required String details, // 예: "페이스 6:30/km 유지"
  }) = _WeekPlan;

  factory WeekPlan.fromJson(Map<String, dynamic> json) =>
    _$WeekPlanFromJson(json);
}

// 하위 모델 2: Action Plan
@freezed
class ActionItem with _$ActionItem {
  @JsonSerializable()
  const factory ActionItem({
    required String id,
    required int priority, // 1-5
    required String task, // 예: "월/수/금 5km 러닝"
  }) = _ActionItem;

  factory ActionItem.fromJson(Map<String, dynamic> json) =>
    _$ActionItemFromJson(json);
}

```

이 모델들이 준비되면, Leader Agent의 Output Structure¹에 명시된 JSON 구조가 Dart 클래스로 정확히 매핑됩니다.

표 1: Leader Agent JSON-Dart 모델 매핑 테이블

JSON Key (LeaderAgentResult)	예시 JSON 유형	Dart Type	fromJson 자동화 처리
gapAnalysis	String	String	json['gapAnalysis'] as String
weeklyRoadmap	List<Object>	List<WeekPlan>	(json as List<dynamic>).map((e) => WeekPlan.fromJson(e as Map<String, dynamic>)).toList()
trendVisualization	Object	Map<String, dynamic>	json['trendVisualization'] as Map<String, dynamic>
actionPlan	List<Object>	List<ActionItem>	(json['actionPlan'] as List<dynamic>).map((e) => ActionItem.fromJson(e as Map<String, dynamic>)).toList()
fallback	bool	bool	json['fallback'] as bool??? false

III. 2단계: 데이터 수신 (Data Reception) - SSE와 Stream을 통한 점진적 응답 구현

모델이 정의되었으므로, 이제 이 모델 객체를 순차적으로 전달하는 데이터 파이프라인을 구축해야 합니다.

아키텍처 제안: 3회 **await** 대신 단일 **SSE(Server-Sent Events)** 연결

¹(p. 19)의 MultiAgentOrchestrator 구현 예시는 3개의 에이전트를 순차적으로 **await**합니다.

Dart

// (p. 19)의 비효율적인 접근 방식

```
final expertAnalysis = await _executeExpertAgent(runningData);  
final emotionalFeedback = await _executeCoachAgent(expertAnalysis);  
final finalResult = await _executeLeaderAgent(expertAnalysis, emotionalFeedback);
```

이 방식은 ¹(p. 18)의 "3-Stage Sequential Execution"을 구현하지만, 클라이언트-서버 간에 3번의 개별 HTTP 왕복(roundtrip)을 유발합니다. 이는 네트워크 지연 시간을 누적시키고 서버 부하를 가중시킵니다.

"점진적 UI"라는 목표를 달성하기 위한 더 효율적이고 현대적인 아키텍처는 단일 **HTTP** 스트리밍 연결을 사용하는 것입니다.

****Server-Sent Events (SSE)****는 이 시나리오에 완벽하게 부합합니다.¹² SSE는 AI 모델의 점진적 토큰 출력¹²을 위해 설계되었으며, 단일 HTTP 연결을 통해 서버가 클라이언트에게 여러 개의 데이터(이 경우, 3개의 JSON 객체)를 순차적으로 푸시(push)할 수 있게 합니다.¹²

백엔드(예: FastAPI)는 `StreamingResponse`¹⁵를 사용하여 각 에이전트의 완료 시점에 JSON 청크(chunk)를 스트리밍할 수 있습니다.¹⁵ Flutter 클라이언트는 `http` 패키지¹⁸ 또는 `flutter_client_sse`¹⁹와 같은 라이브러리를 사용하여 이 스트림을 `Stream` 객체로 수신할 수 있습니다.

구현: MultiAgentOrchestrator 스트림

1

¹(p. 19)의 `Future<AchievementAnalysisModel> performAnalysis` 시그니처를 `Stream<IAgentResult> performAnalysis`로 변경합니다. 이 스트림은 SSE 연결을 수신하고, `event: expert_result`, `event: coach_result`, `event: leader_result` 같은 이벤트가 도착할 때마다¹⁶

`data: {...}` 필드의 JSON 문자열을 `jsonDecode`⁶로 파싱한 후, 1단계에서 정의한 Dart 모델로 변환하여 `yield`합니다.

다음은 `http` 패키지를 사용한 SSE 스트림 수신 및 파싱 예제입니다.

Dart

```
// lib/features/goals/services/multi_agent_orchestrator.dart
import 'dart:convert';
import 'package:http/http.dart' as http; //
import 'package:flutter_riverpod/flutter_riverpod.dart';
// 1단계에서 정의한 모델들 импорт
import 'package:sherpa_app/features/analysis/models/agent_response.model.dart';
import 'package:sherpa_app/features/analysis/models/leader_agent.model.dart';

class MultiAgentOrchestrator {
  final Ref _ref;
  MultiAgentOrchestrator(this._ref);

  // (p. 19)의 Future를 Stream으로 변경
  Stream<IAgentResult> performAnalysis(Map<String, dynamic> runningData) async* {
    final client = http.Client();
    try {
      // 1. SSE 엔드포인트에 스트림 연결 요청
      final request = http.Request(
        'POST', // 또는 'GET', 서버 구현에 따름
        Uri.parse('https://api.sherpa.app/v2/analysis/stream'),
      );
      request.headers['Accept'] = 'text/event-stream'; //
      request.body = jsonEncode(runningData);

      final response = await client.send(request); //

      // 2. 스트림 수신 및 점진적 파싱
      await for (final chunk in response.stream.transform(utf8.decoder)) {
        // SSE 데이터 형식(event: / data:) 파싱
        // 실제 구현에서는 더 견고한 SSE 파서가 필요
        if (chunk.startsWith('event: expert_result')) {
          final jsonData = chunk.split('data: ')[1];
          // 1차 JSON 파싱 및 Dart 모델 변환
```



```

    final result = ExpertAnalysisResult.fromJson(jsonDecode(jsonData));
    yield result; // Stage 1 UI 업데이트

  } else if (chunk.startsWith('event: coach_result')) {
    final jsonData = chunk.split('data: ')[1];
    final result = EmotionalCoachResult.fromJson(jsonDecode(jsonData));
    yield result; // Stage 2 UI 업데이트

  } else if (chunk.startsWith('event: leader_result')) {
    final jsonData = chunk.split('data: ')[1];
    // 2차(중첩) JSON 파싱 및 Dart 모델 변환
    final result = LeaderAgentResult.fromJson(jsonDecode(jsonData));
    yield result; // Stage 3 UI 업데이트
  }
}

} catch (e) {
  // (p. 25)의 폴백 전략 실행
  yield* _handleStreamError(e);
} finally {
  client.close();
}
}

// (p. 25)의 "Expert fails" 폴백 시나리오
Stream<IAgentResult> _handleStreamError(Object e) async* {
  yield ExpertAnalysisResult(
    patternEvaluation: '데이터 분석 중 일시적 오류가 발생했습니다.',
    strengths: ['총 러닝 일수 확인'],
    improvements: ['AI 분석 실패. 다시 시도해주세요.'],
    trendAnalysis: '트렌드 분석 실패',
    statistics: {},
    fallback: true,
  );
  //... Coach, Leader의 기본 폴백 데이터도 yield
}
}

```

IV. 3단계: 상태 관리 (State Management) - StreamBuilder로 스트림과 UI 바인딩

이제 `Stream<IAgentResult>` 데이터 파이프라인이 준비되었습니다. 이 스트림을 `AiAnalysisScreen`¹에 연결하여 ¹(p. 23-24)에 명시된 "점진적 디스플레이"를 구현합니다.

¹(p. 23)의 `_performMultiAgentAnalysis` 메서드는 `setState`와 `_currentStage` 변수를 수동으로 관리하도록 제안되었으나, 이는 `StreamBuilder` 위젯을 사용할 경우 불필요하며 더 복잡한 방식입니다. `StreamBuilder`는 스트림의 `ConnectionState`에 따라 로딩, 에러, 데이터 상태를 자동으로 처리해줍니다.²

상태 누적 (State Accumulation)

`StreamBuilder`는 스트림의 마지막 스냅샷만 기억합니다. 하지만 ¹(p. 24)의 요구사항은 1단계(Expert), 2단계(Coach), 3단계(Leader)의 결과가 누적되어 순차적으로 표시되는 것입니다.

이를 위해, `StreamBuilder`가 감싸는 위젯을 `StatefulWidget`으로 만들고, `builder` 콜백이 호출될 때마다 수신된 데이터를 `List<IAgentResult>`에 추가하여 상태를 누적합니다.

Dart

```
// lib/features/goals/presentation/screens/ai_analysis_screen.dart
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
//... (Orchestrator 및 Models 임포트)

// AiAnalysisScreen을 StatefulWidget으로 변경
class AiAnalysisScreen extends ConsumerStatefulWidget {
  const AiAnalysisScreen({Key? key}) : super(key: key);
  @override
  _AiAnalysisScreenState createState() => _AiAnalysisScreenState();
}

class _AiAnalysisScreenState extends ConsumerState<AiAnalysisScreen> {
  // 3단계의 결과를 누적할 리스트
  final List<IAgentResult> _results =;
```

```
// Orchestrator에서 반환된 스트림
```

```
late final Stream<IAgentResult> _analysisStream;
```

```
// 오류 또는 완료 상태 저장
```

```
AsyncSnapshot<IAgentResult> _snapshot = AsyncSnapshot.nothing();
```

```
@override
```

```
void initState() {
```

```
  super.initState();
```

```
  _performMultiAgentAnalysis();
```

```
}
```

```
void _performMultiAgentAnalysis() {
```

```
  final orchestrator = ref.read(orchestratorProvider);
```

```
  // 실제로는 사용자 데이터(runningData)를 전달해야 함
```

```
  final runningData = ref.read(runningDataProvider);
```

```
  setState(() {
```

```
    _analysisStream = orchestrator.performAnalysis(runningData);
```

```
  });
```

```
}
```

```
@override
```

```
Widget build(BuildContext context) {
```

```
  return Scaffold(
```

```
    appBar: AppBar(title: Text("AI 분석 결과")),
```

```
    body: StreamBuilder<IAgentResult>(
```

```
      stream: _analysisStream,
```

```
      builder: (context, snapshot) {
```

```
        _snapshot = snapshot; // 최종 상태 저장
```

```
        // 1. 스트림 활성화 및 데이터 수신 시
```

```
        if (snapshot.hasData) {
```

```
          // 중복 방지를 위해 이미 수신된 타입인지 확인 후 추가
```

```
          if (!_results.any((res) => res.runtimeType == snapshot.data.runtimeType)) {
```

```
            _results.add(snapshot.data!);
```

```
          }
```

```
        }
```

```
        // 2. 로딩 상태 처리
```

```
        // _results가 비어있고, 아직 완료/에러가 아니면 로딩 중
```

```
        if (_results.isEmpty &&
```

```
            snapshot.connectionState == ConnectionState.waiting) {
```

```
        return Center(child: CircularProgressIndicator()); // , p. 2 LoadingAnimation
    }
```

```
    // 3. 오류 상태 처리
    if (snapshot.hasError) {
        return Center(child: Text('AI 분석 중 오류가 발생했습니다: ${snapshot.error}'));
    }
```

```
    // 4. 데이터 기반 점진적 빌드
    // _results 리스트에 누적된 데이터를 기반으로 UI를 빌드
    return _buildProgressiveDisplay(_results, snapshot.connectionState);
  },
),
);
}
```

// (p. 23-24)의 의도를 StreamBuilder 데이터로 재구성

```
Widget _buildProgressiveDisplay(List<IAgentResult> results, ConnectionState state) {
    // 각 타입의 결과를 찾습니다.
```

```
    final expertResult = results.whereType<ExpertAnalysisResult>().firstOrNull;
    final coachResult = results.whereType<EmotionalCoachResult>().firstOrNull;
    final leaderResult = results.whereType<LeaderAgentResult>().firstOrNull;
```

```
    return ListView( // Column 대신 스크롤 가능한 ListView 사용
        padding: const EdgeInsets.all(16.0),
        children:
            if (expertResult!= null)
                _buildStageCard(
                    title: '운동 전문가 분석',
                    contentWidget: _buildExpertUI(expertResult),
                    isComplete: coachResult!= null, // 2단계가 오면 '완료' 표시
                ),
```

// Stage 2: 감성 코치 피드백

```
            if (coachResult!= null)
                _buildStageCard(
                    title: '감성 코치의 메시지',
                    contentWidget: _buildCoachUI(coachResult),
                    isComplete: leaderResult!= null, // 3단계가 오면 '완료' 표시
                ),
```

// Stage 3: 최종 종합

```
            if (leaderResult!= null)
```

```

    _buildStageCard(
      title: '목표 달성 로드맵',
      contentWidget: _buildLeaderUI(leaderResult), // V 섹션에서 구현
      isComplete: state == ConnectionState.done,
    ),

    // 스트림이 아직 끝나지 않았다면 하단에 로딩 표시
    if (state == ConnectionState.active)
      Padding(
        padding: const EdgeInsets.all(16.0),
        child: Center(child: CircularProgressIndicator()),
      ),
  ],
);
}

//... _buildStageCard, _buildExpertUI, _buildCoachUI, _buildLeaderUI 위젯 구현...
// (V 섹션에서 _buildLeaderUI 상세 구현)
}

```

V. 4단계: UI 렌더링 (UI Rendering) - 3대 핵심 JSON 컴포넌트 시각화

이 섹션은 `_buildLeaderUI(LeaderAgentResult result)` 메서드를 구현하며, 이는 "JSON을 어떻게 UI로 전환하는가?"라는 클라이언트의 질문에 대한 가장 구체적인 답변입니다.¹(p. 17)의 "Visualization Components" 요구사항을 충족시킵니다.

V-A. trendVisualization (트렌드 시각화) 구현

- 요구사항: "Trend Graph: 최근 2개월 거리/페이스 변화".¹
LeaderAgentResult.trendVisualization (Map<String, dynamic>) 데이터를 사용합니다.
- 기술 스택: 2025년 기준 Flutter 커뮤니티에서 가장 선호되는 차트 라이브러리인 `fl_chart`를 사용합니다.²³
- 핵심 변환: `fl_chart`의 `LineChart`²⁵는 데이터 소스로 `List<FlSpot>` 객체를 요구합니다.²⁷
따라서 핵심 과제는 LeaderAgentResult의 trendVisualization 맵에서 데이터 포인트를

추출하여 List<FISpot>으로 변환하는 것입니다.²⁸

Dart

```
// lib/features/analysis/widgets/trend_chart.widget.dart
import 'package:fl_chart/fl_chart.dart'; // [23, 24, 39]
import 'package:flutter/material.dart';

class TrendLineChart extends StatelessWidget {
  // 1단계 모델링에서 정의한 Map<String, dynamic> 타입을 받음
  final Map<String, dynamic> chartData;

  const TrendLineChart({Key? key, required this.chartData}) : super(key: key);

  // JSON List<Map>을 List<FISpot>으로 변환
  List<FISpot> _getSpots() {
    // JSON 구조가 {"type": "line_chart", "data": [{"x": 0, "y": 6.5},...]}라고 가정
    final List<dynamic> dataPoints = chartData['data']??;
    List<FISpot> spots =;
    for (var i = 0; i < dataPoints.length; i++) {
      try {
        final point = dataPoints[i] as Map<String, dynamic>;
        // JSON의 'x', 'y' 키를 FISpot(x, y)로 매핑
        spots.add(FISpot(
          (point['x'] as num).toDouble(),
          (point['y'] as num).toDouble(),
        ));
      } catch (e) {
        // 데이터 파싱 오류 처리
        debugPrint("Error parsing chart spot: $e");
      }
    }
    // 데이터가 없는 경우 빈 차트 대신 기본값 표시
    if (spots.isEmpty) {
      return;
    }
    return spots;
  }

  @override
```

```

Widget build(BuildContext context) {
  return AspectRatio(
    aspectRatio: 1.7,
    child: LineChart(
      LineChartData(
        //.....[26]
        gridData: FlGridData(show: true),
        titlesData: FITitlesData(
          show: true,
          bottomTitles: AxisTitles(sideTitles: SideTitles(showTitles: true)), // X축
          leftTitles: AxisTitles(sideTitles: SideTitles(showTitles: true)), // Y축
        ),
        borderData: FlBorderData(show: true),

        // 핵심 데이터 바인딩 [26, 40]
        lineBarsData:
          // 라인 아래 영역 채우기
          belowBarData: BarAreaData(
            show: true,
            color: Colors.blue.withOpacity(0.1),
          ),
        ),
      ],
    ),
  );
}

```

V-B. weeklyRoadmap (주간 로드맵) 구현

- 요구사항: "Weekly Milestones (카드 형식)".¹ LeaderAgentResult.weeklyRoadmap (List<WeekPlan>) 데이터를 사용합니다.
- 기술 스택: ListView.builder³⁰ 및 Card.³⁰
- 핵심 변환: ¹(p. 17)에서 "카드 형식"을 명시했으므로, List<WeekPlan>의 각 WeekPlan 객체를 Card 위젯으로 감싼 ListTile로 변환합니다. ListView.builder는 4주, 8주 등 동적인 주(week) 길이에도 UI 코드 변경 없이 대응할 수 있게 합니다.³⁰

Dart

```
// lib/features/analysis/widgets/roadmap_list.widget.dart
import 'package:flutter/material.dart';
import 'package:sherpa_app/features/analysis/models/leader_agent.model.dart';

class WeeklyRoadmapList extends StatelessWidget {
  final List<WeekPlan> roadmap;
  const WeeklyRoadmapList({Key? key, required this.roadmap}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    //
    return ListView.builder(
      shrinkWrap: true, // 부모 위젯(Column) 내부에서 스크롤 충돌 방지
      physics: NeverScrollableScrollPhysics(), // 부모 스크롤 사용
      itemCount: roadmap.length,
      itemBuilder: (context, index) {
        final plan = roadmap[index];
        return Card( //
          margin: const EdgeInsets.symmetric(vertical: 8.0),
          child: ListTile(
            leading: CircleAvatar(
              child: Text(
                '${plan.week}', // 주차 표시
                style: TextStyle(fontWeight: FontWeight.bold),
              ),
            ),
            title: Text(plan.summary), // 예: "주 3회 5km"
            subtitle: Text(plan.details), // 예: "페이스 6:30/km 유지"
          ),
        );
      },
    );
  }
}
```

V-C. actionPlan (다음 주 실천 계획) 구현

- 요구사항: "Action Checklist".¹ LeaderAgentResult.actionPlan (List<ActionItem>) 데이터를 사용합니다.
- 기술 스택: CheckboxListTile.³⁴
- 핵심 변환: 이 UI는 단순한 텍스트 표시가 아니라 사용자의 상호작용(체크)을 처리해야 합니다.³⁵ 따라서 이 위젯은 자체적인 상태를 가지는 StatefulWidget이어야 합니다. List<ActionItem>의 각 항목 ID를 키로, 체크 여부를 값으로 하는 Map<String, bool>을 사용하여 각 항목의 상태를 관리하는 것이 가장 효율적입니다.³⁷

Dart

```
// lib/features/analysis/widgets/action_checklist.widget.dart
import 'package:flutter/material.dart';
import 'package:sherpa_app/features/analysis/models/leader_agent.model.dart';

class ActionChecklist extends StatefulWidget {
  final List<ActionItem> actionPlan;
  const ActionChecklist({Key? key, required this.actionPlan}) : super(key: key);
  @override
  _ActionChecklistState createState() => _ActionChecklistState();
}

class _ActionChecklistState extends State<ActionChecklist> {
  // 패턴: Map을 사용해 동적 체크박스 상태 관리
  final Map<String, bool> _checkedState = {};

  @override
  Widget build(BuildContext context) {
    return ListView.builder(
      shrinkWrap: true,
      physics: NeverScrollableScrollPhysics(),
      itemCount: widget.actionPlan.length,
      itemBuilder: (context, index) {
        final item = widget.actionPlan[index];

        // (p.17)의 우선순위(1-5)를 시각적으로 표시
        final priorityPrefix = "P${item.priority} ";

        //
```

```

return CheckboxListTile(
  title: Text(priorityPrefix + item.task),
  value: _checkedState[item.id]?? false,
  onChanged: (bool? newValue) {
    setState(() { // [35, 37]
      _checkedState[item.id] = newValue?? false;
    });
    // TODO: 사용자의 체크 상태를 로컬 DB나 SharedPreferences에 저장
  },
);
},
);
}
}

```

_buildLeaderUI 최종 통합

이제 세 개의 핵심 위젯을 **_buildLeaderUI** 메서드 내에서 조립하여 3단계 UI를 완성합니다.

Dart

```

// AiAnalysisScreenState 내부에 구현
Widget _buildLeaderUI(LeaderAgentResult result) {
  // (p. 25)의 폴백(Fallback) 시나리오 처리
  if (result.fallback) {
    return Text("로드맵을 생성하는 중 오류가 발생했습니다. 기본 계획을 표시합니다.");
    //... _generateBasicRoadmap(user)
  }

  // 정상적인 시각화 UI
  return Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children:
  );
}

```

VI. 견고성 확보: Agent-Level 폴백(Fallback) 전략의 UI 구현

¹(p. 25)는 "Agent-Level Fallback Strategy"를 정의하며, 이는 UI의 견고성을 위해 반드시 구현되어야 합니다. `StreamBuilder`와 `IAgentResult` 인터페이스의 `fallback: true` 플래그는 이 전략을 효과적으로 구현할 수 있게 합니다.

- 시나리오 1: "Coach fails" ¹
 - 흐름: `Orchestrator`(III단계)가 `_executeCoachAgent`에서 오류를 catch합니다.
 - `Orchestrator`는 ¹(p. 25)의 "Fallback to generic encouragement"에 따라 `EmotionalCoachResult(encouragement: '꾸준히 노력하시는 모습이...'`, `fallback: true`) 객체를 yield합니다.
 - `StreamBuilder`(IV단계)가 이 `fallback: true` 객체를 수신하고 `_results` 리스트에 추가합니다.
 - `_buildProgressiveDisplay`의 `_buildCoachUI` 함수는 `coachResult.fallback == true`를 감지하고, 맞춤형 메시지 대신 이 일반 격려 메시지를 렌더링합니다. 사용자는 부분적인 성공(Partial Success)을 경험하게 됩니다.
- 시나리오 2: "Expert fails" ¹
 - 흐름: 이는 가장 치명적인 시나리오입니다. ¹(p. 18)의 의존성 다이어그램에서 보듯이, `Expert Agent`(Agent 1)가 실패하면 `Coach`(Agent 2)와 `Leader`(Agent 3)가 입력을 받지 못해 연쇄적으로 실패합니다.
 - 구현: `Orchestrator`는 `ExpertAnalysisResult.fallback()`을 yield합니다 (III단계의 `_handleStreamError` 참조).
 - UI의 `StreamBuilder`는 `expertResult.fallback == true`를 감지하고, "데이터 분석 중 일시적 오류가 발생했습니다." ¹라는 메시지를 표시합니다. `Coach` 및 `Leader` 단계는 "기본 통계" 및 "기본 로드맵"으로 대체 렌더링되거나¹, 치명적 오류로 간주하여 분석을 중단하고 사용자에게 포인트 환불¹을 안내할 수 있습니다.

VII. 결론 및 아키텍처 전환 권장 사항

클라이언트의 질문인 "JSON 데이터로 반환이 되면 어떻게 또 UI로 전환을 하는데?"에 대한 아키텍처 수준의 답변은 다음과 같은 4단계의 유기적인 구현 전략으로 요약됩니다.

1. 모델링 (Modeling): `freezed`와 `json_serializable`을 사용하여 서버의 멀티 에이전트 JSON 스키마¹와 1:1로 매핑되는 타입-안전한 Dart 모델(`IAgentResult` 및 그 구현체)을

설계합니다.

2. 수신 (**Reception**): MultiAgentOrchestrator가 Future가 아닌 Stream을 반환하도록 재설계합니다. 이 스트림은 단일 HTTP 연결(SSE 권장)¹²을 통해 백엔드로부터 3단계의 JSON 응답을 순차적으로 수신하고, jsonDecode²¹ 및 fromJson 팩토리 메서드를 통해 Dart 모델 객체로 변환하여 yield합니다.
3. 바인딩 (**Binding**): AiAnalysisScreen¹에서 StreamBuilder²를 사용하여 UI 위젯 트리와 2단계의 데이터 스트림을 연결합니다. StreamBuilder 내부에서 상태를 누적(List<IAgentResult>)하여¹(p. 23-24)의 "점진적 디스플레이" 요구사항을 충족합니다.
4. 렌더링 (**Rendering**): StreamBuilder의 builder 콜백 내에서 수신된 JSON 모델(LeaderAgentResult)을 fl_chart (트렌드)²³, ListView (로드맵)³⁰, CheckboxListTile (실천 계획)³⁵ 등 구체적인 Flutter 위젯으로 변환하여¹(p. 17)에서 요구하는 "시각화된" UI를 최종적으로 완성합니다.

이 아키텍처는¹(p. 22)에서 제시된 "40-50% 더 빠른" 체감 속도, "실시간 피드백" UX, 그리고 "부분적 성공(Partial Success)"을 통한 안정성 향상 등 멀티 에이전트 아키텍처의 모든 이점을 실현하는 핵심적인 Flutter 구현 전략입니다.

참고 자료

1. AI_ANALYSIS_SYSTEM_ARCHITECTURE.pdf
2. Mastering StreamBuilder in Flutter — With Real Example and Pro Tips | by Raj Jani, 11월 12, 2025에 액세스, <https://medium.com/easy-flutter/mastering-streambuilder-in-flutter-with-real-example-and-pro-tips-6c01c656e1d3>
3. How to Work with Streams in Flutter for Real-Time Updates | by REIT monero | Medium, 11월 12, 2025에 액세스, <https://medium.com/@juricavoda/how-to-work-with-streams-in-flutter-for-real-time-updates-5c47dd3ce9eb>
4. StreamBuilder class - widgets library - Dart API - Flutter, 11월 12, 2025에 액세스, <https://api.flutter.dev/flutter/widgets/StreamBuilder-class.html>
5. Flutter - StreamBuilder Widget - GeeksforGeeks, 11월 12, 2025에 액세스, <https://www.geeksforgeeks.org/flutter/flutter-streambuilder-widget/>
6. How to Parse JSON in Dart/Flutter: The Ultimate Guide - Code With Andrea, 11월 12, 2025에 액세스, <https://codewithandrea.com/articles/parse-json-dart/>
7. Using JSON Serializable in Flutter: A Quick Start Guide - Bugsee, 11월 12, 2025에 액세스, <https://bugsee.com/flutter/flutter-json-serializable/>
8. How to use json_serializable package for complex response models - Flutter, 11월 12, 2025에 액세스, <https://stackoverflow.com/questions/72983945/how-to-use-json-serializable-package-for-complex-response-models-flutter>
9. Handling Large JSON Data Efficiently with json_serializable and isolates in Flutter, 11월 12, 2025에 액세스, <https://vibe-studio.ai/insights/handling-large-json-data-efficiently-with-json-serializable-and-isolates-in-flutter>

10. Parsing complex JSON in Flutter - Medium, 11월 12, 2025에 액세스,
<https://medium.com/flutter-community/parsing-complex-json-in-flutter-747c46655f51>
11. How to parse Nested JSON in flutter? - Stack Overflow, 11월 12, 2025에 액세스,
<https://stackoverflow.com/questions/71382868/how-to-parse-nested-json-in-flutter>
12. Top Flutter Server-Sent Events (SSE) packages, 11월 12, 2025에 액세스,
<https://fluttergems.dev/server-sent-events/>
13. Real-Time Data Streaming with Server-Sent Events (SSE) | by Serif Colakel | Medium, 11월 12, 2025에 액세스,
<https://medium.com/@serifcolakel/real-time-data-streaming-with-server-sent-events-sse-9424c933e094>
14. Server Sent Events(SSE) with Flutter | by Vinay Shankri | Flutter Community | Medium, 11월 12, 2025에 액세스,
<https://medium.com/flutter-community/server-sent-events-sse-with-flutter-cf331f978b4f>
15. Streaming APIs for Beginners: Python, FastAPI, and Async Generators | by Okan Yenigün, 11월 12, 2025에 액세스,
<https://python.plainenglish.io/streaming-apis-for-beginners-python-fastapi-and-async-generators-848b73a8fc06>
16. How to stream JSON data using Server-Sent Events - Stack Overflow, 11월 12, 2025에 액세스,
<https://stackoverflow.com/questions/76486517/how-to-stream-json-data-using-server-sent-events>
17. How to Stream Structured JSON Output from LLMs Using FastAPI and PydanticAI, 11월 12, 2025에 액세스,
<https://python.plainenglish.io/how-to-stream-structured-json-output-from-llms-using-fastapi-and-pydanticai-c1daca666ca6>
18. How to stream responses of a specific API that supports Server-Sent Events (SSE) in flutter?, 11월 12, 2025에 액세스,
<https://stackoverflow.com/questions/76502116/how-to-stream-responses-of-a-specific-api-that-supports-server-sent-events-sse>
19. flutter_client_sse example | Flutter package - Pub.dev, 11월 12, 2025에 액세스,
https://pub.dev/packages/flutter_client_sse/example
20. ElshiatyTube/flutter_http_sse: Flutter HTTP Server-Sent Events (SSE) client library for receive real-time updates from the server - GitHub, 11월 12, 2025에 액세스,
https://github.com/ElshiatyTube/flutter_http_sse
21. JSON and serialization - Flutter documentation, 11월 12, 2025에 액세스,
<https://docs.flutter.dev/data-and-backend/serialization/json>
22. jsonDecode function - dart:convert library - Dart API - Flutter, 11월 12, 2025에 액세스,
https://api.flutter.dev/flutter/dart_convert/jsonDecode.html
23. Top Flutter Charts, Plots, Visualization packages for line chart, bar chart, radial chart, pie chart, sparkline, speedometer & more | Flutter Gems, 11월 12, 2025에 액세스,
<https://fluttergems.dev/plots-visualization/>
24. Creating Beautiful Charts in Flutter with fl_chart | by Sanjay Adhikari - Medium,

- 11월 12, 2025에 액세스,
https://medium.com/@sanjayadhikari_95964/creating-beautiful-charts-in-flutter-with-fl-chart-c942f4abd69c
25. Flutter Line Chart Tutorial - FLChart Line Graph Guide - YouTube, 11월 12, 2025에 액세스, <https://www.youtube.com/watch?v=7jfnJclkmhU>
26. Flutter Charts Tutorial: 6 Types with Code Samples, 11월 12, 2025에 액세스, <https://mobisoftinfotech.com/resources/blog/app-development/flutter-charts-tutorial-6-types-with-code-samples>
27. Flutter: Design this type of FL_CHART line chart - Stack Overflow, 11월 12, 2025에 액세스,
<https://stackoverflow.com/questions/66865312/flutter-design-this-type-of-fl-chart-line-chart>
28. How to assign an array to spots in fl_chart in Flutter - Stack Overflow, 11월 12, 2025에 액세스,
<https://stackoverflow.com/questions/60281061/how-to-assign-an-array-to-spots-in-fl-chart-in-flutter>
29. How to Visualize Data in Flutter with fl_chart and Back4App - Tutorials, 11월 12, 2025에 액세스,
<https://www.back4app.com/tutorials/how-to-visualize-data-in-flutter-with-fl-chart-and-back4app>
30. Flutter read data from local JSON files | ListView - Dbestech, 11월 12, 2025에 액세스,
<https://www.dbestech.com/tutorials/flutter-read-data-from-local-json-files>
31. Load JSON string into Flutter list view dynamic (without loading into a model), 11월 12, 2025에 액세스,
<https://stackoverflow.com/questions/73864326/load-json-string-into-flutter-list-view-dynamic-without-loading-into-a-model>
32. Flutter Parse Json Data in ListView - YouTube, 11월 12, 2025에 액세스,
<https://www.youtube.com/watch?v=KCYKYe1Ts1o>
33. Flutter: Building ListView with JSON Data | HTTP GET - YouTube, 11월 12, 2025에 액세스, <https://www.youtube.com/watch?v=1OEhlpBIZXM>
34. CheckboxListTile class - material library - Dart API - Flutter, 11월 12, 2025에 액세스, <https://api.flutter.dev/flutter/material/CheckboxListTile-class.html>
35. How to create a checkbox list Using a CheckboxListTile - ApparenceKit, 11월 12, 2025에 액세스,
<https://apparencekit.dev/flutter-tips/how-to-create-checkbox-list-using-checkboxlisttile-flutter/>
36. CheckboxListTile (Flutter Widget of the Week) - YouTube, 11월 12, 2025에 액세스,
<https://www.youtube.com/watch?v=RkSqPAn9szs>
37. How can i add multiple expand tile with checkbox list tile using Json in flutter, 11월 12, 2025에 액세스,
<https://stackoverflow.com/questions/57721168/how-can-i-add-multiple-expand-tile-with-checkbox-list-tile-using-json-in-flutter>
38. How to setup factory album for nested json values - Stack Overflow, 11월 12, 2025에 액세스,

<https://stackoverflow.com/questions/70467299/how-to-setup-factory-album-for-nested-json-values>